COLLEGE PROJECT DOCUMENTATION

CareConnect

Patient Record & Healthcare Management System

Comprehensive Software Requirements & System Modules Specification

---

PROJECT TYPE
Full Stack Enterprise Web Application

BACKEND FRAMEWORK
Java & Spring Boot 3.x (Spring Web, Data JPA, Security)

FRONTEND FRAMEWORK
Angular (TypeScript, RxJS, Modern UI)

DATABASE ENGINE
Oracle DB (Oracle 19c / 21c / 23c XE)

TARGET ACADEMIC LEVEL
B.Tech / B.E. / BCA / MCA Final/Semester Project

DOCUMENTATION VERSION
1.0 (Viva & Evaluation Ready)

1. Executive Summary & Project Objectives

CareConnect is an integrated enterprise-grade Electronic Health Record (EHR) and Patient Management solution designed to digitize outpatient clinical workflows. The software bridges the communication gap between patients, healthcare professionals, and clinic administration by providing secure medical record tracking, digital appointments, and tamper-proof electronic prescriptions.

> **Core Mission:** To replace disorganized physical paper files with a centralized, role-protected digital record system backed by a high-availability Oracle Database and enterprise Spring Boot & Angular architecture.

Key Objectives for Academic Evaluation:

- ✓ **Role-Based Access Control (RBAC):** Strict separation between Administrative, Clinical (Doctor), and Patient operations.
- ✓ **Unified Patient Dossier:** Instant chronological retrieval of past diagnoses, vitals, allergies, and drug histories.
- ✓ **Lifecycle-Driven Appointments:** State machine tracking from initial booking to consultation completion.
- ✓ **Standardized E-Prescriptions:** Legible, digital prescription authoring with dosage, frequency, and instructions.
- ✓ **Enterprise Architecture:** Powered by Java, Spring Boot, Angular components, and Oracle Database persistence.
- ✓ **Interactive OpenAPI / Swagger Documentation:** Direct API demonstration capability for viva panels.

2. System Architecture & Tech Stack

The application follows a decoupled **Client-Server (N-Tier)** architecture adhering to RESTful principles:

```
[ Angular Client (TypeScript/RxJS) ] ──── HTTPS / REST (JSON) ────▶ [ Spring Boot 3 Controller ] | [ Service Layer ] (Business Logic) | [ Spring Data JPA ] (Hibernate) | [ Oracle DB (ojdbc11) ]
```

Tier	Technology	Justification & Role in Project
Client (Frontend)	Angular (TypeScript, HTML5, CSS3)	Robust single-page application framework with strong typing, two-way data binding, dependency injection, and reusable components.
API & Business Logic	Java (JDK 17/21/26), Spring Boot 3.x	Industry-standard enterprise backend. Dependency Injection, high throughput, and robust ecosystem.
Security & Auth	Spring Security + JWT	Stateless authentication using Bearer tokens. Protects medical records from unauthorized access.

Tier	Technology	Justification & Role in Project
Data Persistence	Spring Data JPA (Hibernate)	Entity-relational mapping, automatic schema generation, and query optimization using Oracle Dialect.
Database Engine	Oracle DB (ojdbc11 driver)	High-performance, ACID-compliant relational enterprise database for mission-critical healthcare records and audit trails.
API Documentation	Springdoc OpenAPI (Swagger UI)	Interactive web documentation to test and present all endpoints live to evaluators.

3. Detailed Functional Modules Breakdown

To keep the project clean, manageable, and highly demonstrable during college defense, the system is organized into **4 Essential Functional Modules**:

MODULE 01 User Authentication & Role-Based Access Control (RBAC)

Security Core

Purpose: Secures the system, handles user authentication, and strictly isolates patient privacy and doctor operations based on roles.

Functional Features:

- ✓ **Multi-Role Registration & Login:** Separate onboarding flows for Patients, Doctors, and Receptionist/Admin.
- ✓ **Stateless JWT Security:** Generates a signed JSON Web Token upon successful credential verification.
- ✓ **Role Gatekeepers:** Endpoints are secured using annotations such as `@PreAuthorize("hasRole('DOCTOR')")`.
- ✓ **Password Hashing:** BCrypt algorithm with salted rounds ensures zero plaintext password storage.

Endpoints Covered:

Method	Endpoint	Role	Description
`POST`	`/api/auth/register`	Public	Register a new patient account
`POST`	`/api/auth/login`	Public	Verify credentials & issue JWT token
`GET`	`/api/auth/me`	Authenticated	Retrieve current user profile & role

MODULE 02 Patient Health Records & Demographics (EHR)

Patient & Doctor

Purpose: Stores clinical baseline data, vital statistics, emergency contact details, and chronic health history.

Functional Features:

- ✓ **Patient Profile:** Tracks Date of Birth, Gender, Blood Group (A+, B+, O+, etc.), Contact Info, and Emergency Contact.
- ✓ **Clinical Vitals Tracker:** Records Height, Weight, Blood Pressure, Pulse, and Known Allergies (e.g., Penicillin, Peanuts).

- ✓ **Timeline View:** Shows historical timeline of all past hospital encounters, diagnoses, and lab summaries.
- ✓ **Patient Privacy Protection:** Patients can view only their own records; doctors can view records of assigned patients.

Endpoints Covered:

Method	Endpoint	Role	Description
`GET`	`/api/patients/profile`	Patient	Get logged-in patient's complete medical file
`PUT`	`/api/patients/profile`	Patient	Update personal contact details and vitals
`GET`	`/api/patients/{id}/records`	Doctor/Admin	Doctor retrieves patient's history during consultation

MODULE 03 Appointment Scheduling & Consultation Lifecycle

Interactive Engine

Purpose: Coordinates patient booking requests, doctor schedules, and tracks consultation stages from start to finish.

Appointment Lifecycle State Machine:

```
[ Patient Books ] ──▶ [ PENDING ] ──(Doctor Accepts)──▶ [ CONFIRMED ] |
(Consultation Done) | ▼ [ COMPLETED ] (Prescription linked) *(Also supports
[ CANCELLED ] state with audit reason)*
```

Functional Features:

- ✓ **Doctor Directory & Specialty Search:** Filter doctors by department (General Medicine, Cardiology, Pediatrics, etc.).
- ✓ **Slot Reservation:** Patient selects doctor, preferred date, time slot, and reason for consultation.
- ✓ **Doctor Workstation:** Doctor views calendar list of queued patients with real-time status badges.
- ✓ **One-Click Status Updates:** Doctor can confirm, begin, or complete appointments directly from dashboard.

Endpoints Covered:

Method	Endpoint	Role	Description
`GET`	`/api/doctors`	Public / Auth	List doctors with qualifications, fees & specialty
`POST`	`/api/appointments/book`	Patient	Create appointment booking request
`GET`	`/api/appointments/my`	Patient/Doctor	Get appointments filtered by user role
`PATCH`	`/api/appointments/{id}/status`	Doctor	Update status (CONFIRMED / COMPLETED / CANCELLED)

MODULE 04 Clinical Diagnosis & E-Prescription Generator

Clinical Output

Purpose: Enables doctors to record consultation diagnoses and issue clean, structured digital prescriptions with printable view.

Functional Features:

- ✓ **Structured Medication Entry:** Drug name, dosage (e.g., 500mg), frequency (1-0-1), duration (5 days), and intake instruction (After food).
- ✓ **Clinical Notes & Advice:** Doctor adds clinical observations, dietary suggestions, and recommended lab investigations.
- ✓ **Printable Prescription View:** Clean hospital-header layout ready for browser 1-click printing or PDF export.
- ✓ **Immutable Record:** Once issued and appointment marked completed, prescription is permanently preserved in patient history.

Endpoints Covered:

Method	Endpoint	Role	Description
`POST`	`/api/prescriptions`	Doctor	Create & issue prescription for an appointment
`GET`	`/api/prescriptions/{appointmentId}`	Doctor/Patient	View prescription details for a specific visit
`GET`	`/api/prescriptions/patient/history`	Patient	Patient views all past prescriptions

4. Database Schema (5 Essential Entities)

To avoid over-engineering while maintaining relational integrity, the database uses 5 well-normalized tables:

Table Name	Primary Key	Foreign Keys	Key Columns & Data Types
`users`	`id (BIGINT)`	-	`name (VARCHAR)`, `email (VARCHAR UNIQUE)`, `password (VARCHAR)`, `role (ENUM: ADMIN, DOCTOR, PATIENT)`
`patients`	`id (BIGINT)`	`user_id → users(id)`	`dob (DATE)`, `gender (VARCHAR)`, `blood_group (VARCHAR)`, `phone (VARCHAR)`, `allergies (TEXT)`
`doctors`	`id (BIGINT)`	`user_id → users(id)`	`specialization (VARCHAR)`, `qualification (VARCHAR)`, `consultation_fee (DECIMAL)`, `available_days (VARCHAR)`
`appointments`	`id (BIGINT)`	`patient_id → patients(id)` `doctor_id → doctors(id)`	`appointment_date (DATETIME)`, `reason (TEXT)`, `status (ENUM: PENDING, CONFIRMED, COMPLETED, CANCELLED)`
`prescriptions`	`id (BIGINT)`	`appointment_id → appointments(id)`	`diagnosis (VARCHAR)`, `medicines (JSON / TEXT)`, `doctor_notes (TEXT)`, `created_at (TIMESTAMP)`

5. College Viva Presentation & Demonstration Script

Follow this exact 5-step live execution plan during your project evaluation to score top marks:

1. **Step 1: Admin / System Overview (1 Min)**
   Log into Admin portal. Show the dashboard metrics: Total registered doctors, active patients, and pending appointments. Show database seeding.
2. **Step 2: Patient Registration & Booking (1.5 Min)**
   Register a test patient (e.g., *"John Doe"*). Update health vitals (Blood Group: O+, Allergies: Penicillin). Browse doctor directory, choose Dr. Smith (Cardiologist), and book a slot for today. Status turns to `PENDING`.
3. **Step 3: Doctor Consultation & Prescription (1.5 Min)**
   Log in as Dr. Smith. Show incoming consultation queue. Doctor opens John Doe's profile, reviews past health history, and clicks **"Accept & Start Consultation"**. Status updates to `CONFIRMED`.

4. **Step 4: Prescription Authoring & Printable Output (1 Min)**
   Doctor types Diagnosis: *"Mild Hypertension"*. Adds medication: *"Amlodipine 5mg - Once Daily - 15 Days"*. Clicks **"Issue Prescription & Complete Visit"**.
5. **Step 5: Patient Receipt & Swagger API Verification (1 Min)**
   Switch to Patient view: show the completed visit with the generated prescription and click **"Print Prescription"**. Open `http://localhost:8080/swagger-ui.html` to show backend REST endpoints to the viva examiner.
